

Codeforces 1732A - Bestie

Step 1: What is the operation?

We can choose any index i and perform

```
a[i] = gcd(a[i], i);
```

The cost of this operation is

```
n - i + 1
```

So the costs are

Index	Cost
1	n
2	n-1
...	...
n-1	2
n	1

So the larger the index, the cheaper the operation.

Step 2: First observation

Find

```
G = gcd(all elements)
```

If

```
G == 1
```

then the array is already "good".

Answer = 0

Step 3: What happens after one operation?

Suppose

```
G = gcd(all elements)
```

Every element is divisible by G.

Now we change index i

```
a[i] = gcd(a[i], i);
```

The new overall GCD becomes

```
gcd(G, i)
```

Why?

Before the operation,

```
a[i] = G × k
```

After the operation,

```
gcd(a[i], i)
= gcd(G × k, i)
```

When we compute the GCD of the whole array, every other element still contributes G, so the new common divisor is

```
gcd(G, gcd(a[i], i))
= gcd(G, i)
```

This is the most important observation.

Notice that only the index matters, not the value of `a[i]`.

Step 4: Which indices should we try?

Since

- changing index n costs 1
- changing index n-1 costs 2

we first try the cheapest operations.

Case 1

Modify only the last element.

```
a[n] = gcd(a[n], n);
```

If the new GCD becomes 1

Answer = 1

Case 2

Restore the array.

Modify only the second-last element.

```
a[n-1] = gcd(a[n-1], n-1);
```

If the new GCD becomes 1

Answer = 2

Your doubt

You asked:

"What if changing only the first element (or some middle element) makes the GCD 1? We never check that!"

This is a very good question.

The answer

Suppose

Only index 1 works.

Cost of changing index 1 is

n

But the problem has a mathematical property:

If neither n nor $n-1$ alone works, then changing both always works.

Changing both costs

$1 + 2 = 3$

Since

$3 \leq n$

(for all relevant n), changing index 1 can never give a better answer.

So we don't need to check it.

Why do the last two indices always work?

This is because

$\gcd(n, n-1) = 1$

Consecutive numbers never share any prime factor.

Example

$$9 = 3^2$$

$$10 = 2 \times 5$$

No common prime factor.

Suppose

$$G = 6 = 2 \times 3$$

Changing only 10

$$\gcd(6, 10) = 2$$

Factor 3 disappears.

Changing only 9

$$\gcd(6, 9) = 3$$

Factor 2 disappears.

Changing both

$$\begin{aligned} &\gcd(6, 10, 9) \\ &= \\ &\gcd(\gcd(6, 10), 9) \\ &= \\ &\gcd(2, 9) \\ &= \\ &1 \end{aligned}$$

All prime factors are removed.

Therefore the overall GCD becomes 1.

This is why checking the last two indices is always enough.

Final Algorithm

```
Find GCD of the entire array.

If GCD == 1
    Answer = 0

Else
    Change last element.
    If GCD == 1
        Answer = 1

    Else
        Restore array.
        Change second-last element.
        If GCD == 1
            Answer = 2

    Else
        Answer = 3
```

Time Complexity

Computing GCD of the array takes $O(n)$.

We do it at most 3 times.

Overall:

$O(n)$

Interview intuition (30 seconds)

"First, compute the GCD of the entire array. If it's already 1, no operation is needed. Otherwise, after changing index i , the new overall GCD depends only on $\gcd(\text{currentGCD}, i)$. Since operations on larger indices are cheaper, we first try the last element (cost 1), then the second-last (cost 2). A key mathematical fact is that consecutive indices n and $n-1$ are coprime, so if neither operation alone makes the GCD 1, performing both always removes every remaining common prime factor. Hence the answer is always one of 0, 1, 2, or 3."

Perfect! Let's put everything together in the correct order. This is the explanation I would give in an interview or use to revise before a contest.

Step 1: Understand the operation

For every index i , we have only two choices:

- Don't operate $\rightarrow$ value remains `a[i]`
- Operate $\rightarrow$ value becomes

```
gcd(a[i], i)
```

Notice that operating twice has no effect because

```
gcd(gcd(a[i], i), i) = gcd(a[i], i)
```

So each index has only 2 possible states.

Step 2: What are we trying to find?

The problem asks:

Choose some indices to operate on so that the GCD of the entire array becomes 1 with minimum cost.

The important words are:

- Choose any combination of indices
- GCD of the entire array
- Minimum cost

Step 3: Why do we use subsets?

Since each index has only 2 choices:

- Operate
- Don't Operate

For n indices, the total number of possible combinations is

```
2^n
```

Each combination (subset) represents:

The indices on which we perform the operation.

For example,

```
Subset = {2, 4}
```

means

```
Operate on index 2  
Operate on index 4  
Leave all others unchanged
```

The subset does NOT mean we take the GCD of those indices.

Step 4: What do we check for each subset?

For every subset:

- Build the new array.
- Compute the GCD of the entire new array.
- If the GCD is 1, then this subset is a valid solution.
- Compute its cost.
- Keep the minimum cost.

Step 5: Where is the GCD of the whole array computed?

This line:

```
g = gcd(g, val);
```

Suppose the new array is

```
6 2 15 4
```

Then

```
g = 0  
  
g = gcd(0, 6)    = 6  
g = gcd(6, 2)    = 2  
g = gcd(2, 15)   = 1  
g = gcd(1, 4)    = 1
```

After the loop,

```
g
```

stores

```
gcd(6, 2, 15, 4)
```

which is the GCD of the entire array.

Step 6: Why do we check $g == 1$?

Because the problem asks:

Make the GCD of all numbers equal to 1.

So after processing one subset:

```
if (g == 1)
    ans = min(ans, cost);
```

means:

"This combination of operations works. Compare its cost with the best answer."

Step 7: Why does bitmask work?

Bitmask simply represents one subset.

Example:

```
mask = 1010
```

means

```
Index 1 → Don't Operate
Index 2 → Operate
Index 3 → Don't Operate
Index 4 → Operate
```

So each mask represents one possible final array.

Overall Algorithm

```
Read array
↓
For every subset (mask)
↓
Modify only the selected indices
↓
Compute GCD of the ENTIRE resulting array
↓
If GCD == 1
↓
Compute cost
↓
Keep the minimum cost
```

Time Complexity

There are

```
2^n
```

subsets.

For each subset, we process all n elements.

So,

$$\text{Time} = O(n \times 2^n)$$

Since

$$n \leq 20$$

this is fast enough.

| The Most Important Takeaway ★

The confusion usually comes from the word subset.

Remember this one sentence:

The subset is only used to decide which indices are modified. After applying those modifications, we always compute the GCD of the entire resulting array—not the GCD of the subset.

If you keep this idea in mind, the whole solution becomes straightforward.